



UNIVERSITÀ DEGLI STUDI DI SALERNO
DIPARTIMENTO DI INFORMATICA
DIPARTIMENTO DI ECCELLENZA



CORSO DI LAUREA IN INFORMATICA

Tecnologie Software per il Web

AJAX

PROF. ATTILIO DELLA GRECA

a.a. 2025-2026

AJAX: Asynchronous JavaScript And Xml



AJAX è il sogno di ogni sviluppatore, perché permette di:

- Leggere i dati da un server web dopo che una pagina web è stata caricata.
- Aggiornare una pagina web senza ricaricare la pagina
- Invio dei dati a un server web - in background

https://www.w3schools.com/whatis/whatis_ajax.asp

Un nuovo modello: DHTML

Con DHTML (*client-side JavaScript + HTML + CSS*) il Web inizia a comportarsi come un'applicazione tradizionale, ma con DUE livelli di eventi distinti che convivono nella stessa pagina:



Eventi LOCALI

Gestiti interamente nel browser

- JavaScript intercetta l'evento (es. onClick, onChange)
- Modifica il DOM direttamente nel browser
- Cambiamenti locali, immediati, senza rete
- Esempi: validazione form, mostra/nascondi, animazioni

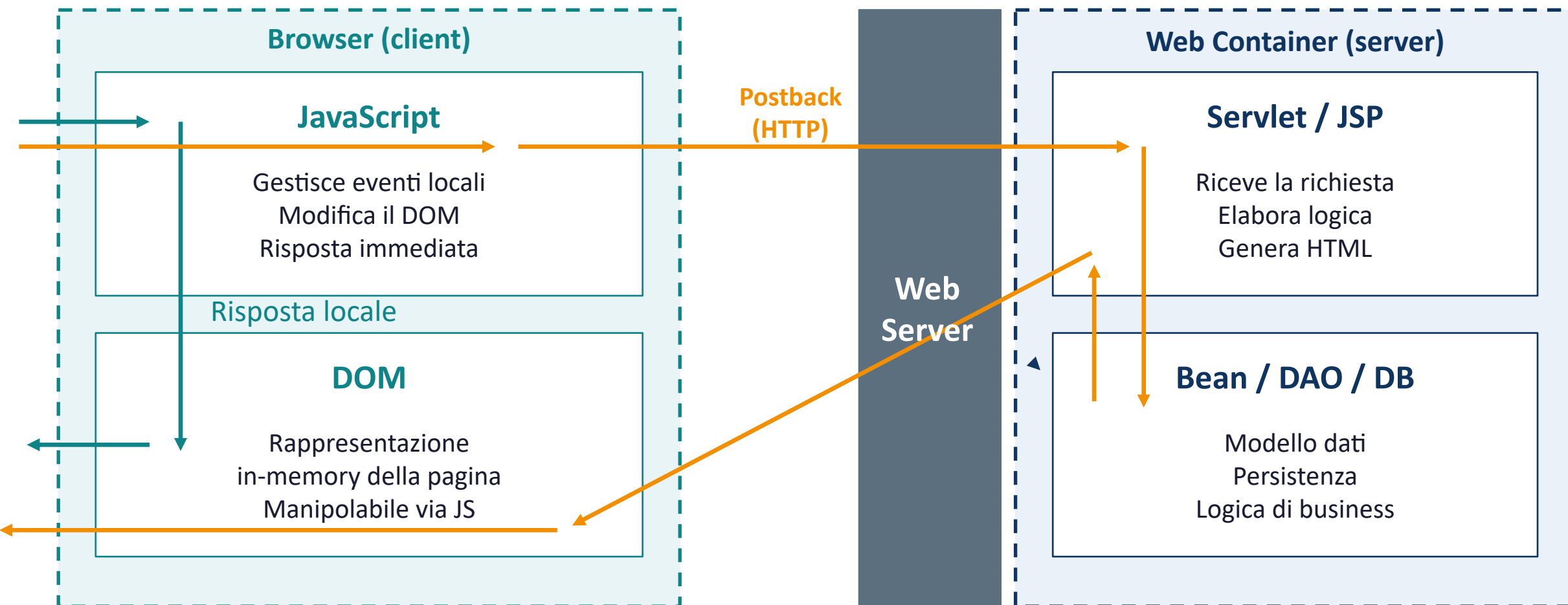


Eventi REMOTI (postback)

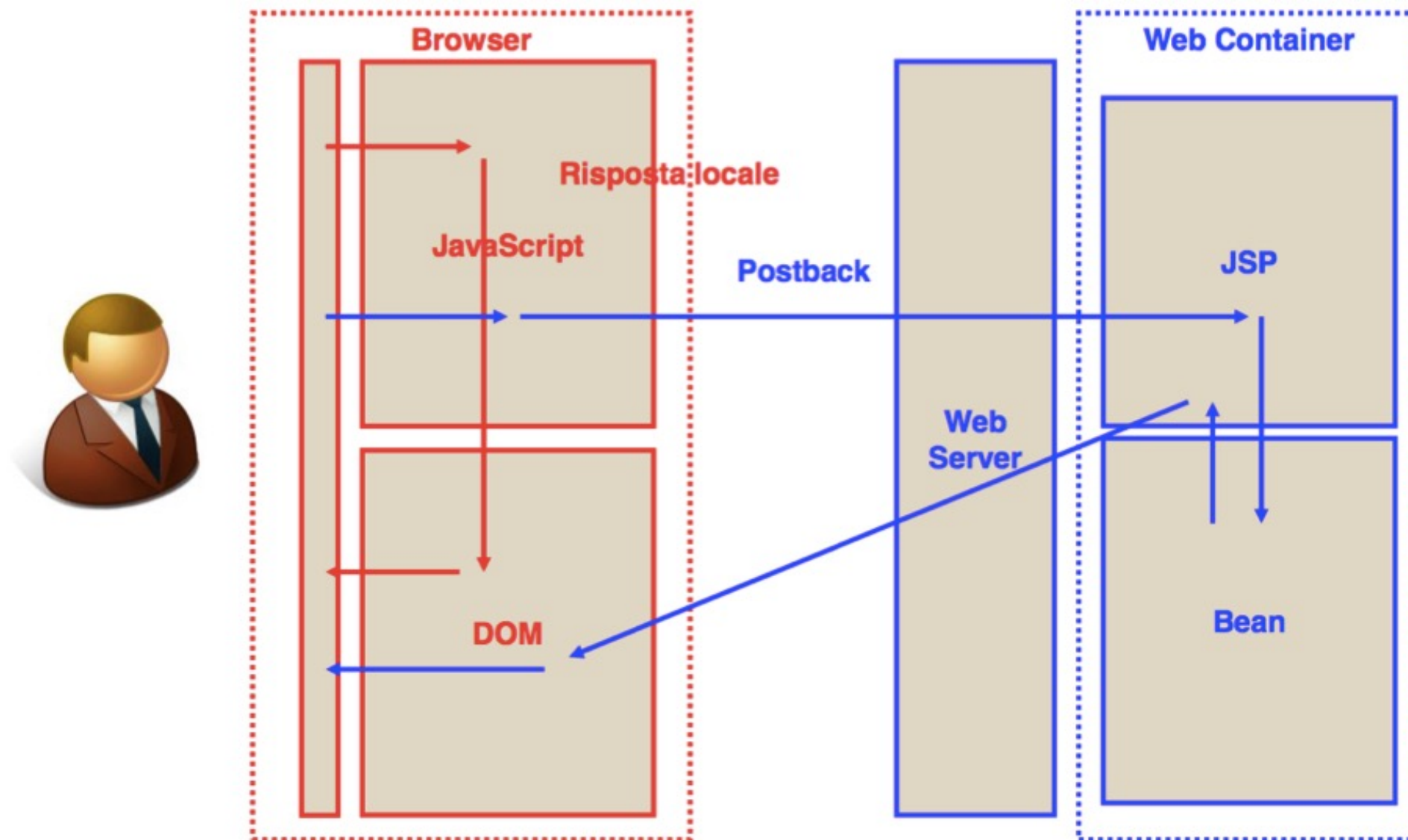
Richiedono il ricaricamento della pagina

- Il browser invia una nuova richiesta HTTP (GET/POST)
- Il server elabora e restituisce una pagina nuova
- La pagina viene completamente ricaricata
- Postback = ricarica pagina come risposta a un evento utente

Architettura del modello a due livelli



Modello a eventi a due livelli



Esempio di postback

Consideriamo un form in cui compaiono due tendine che servono a selezionare il comune di nascita di una persona

- Una con province
- Una con comuni

Si vuole fare in modo che scegliendo la provincia nella prima tendina, nella seconda appaiano solo i comuni di quella provincia

Provincia / Comune di nascita ★

SALERNO

FISCIANO

Esempio di postback

Per realizzare questa interazione si può procedere in questo modo:

1. Si crea una JSP che inserisce nella tendina dei comuni l'elenco di quelli che appartengono alla provincia passata come parametro
2. Si definisce un evento **onChange** collegato all'elemento **select** delle province
3. Lo script collegato ad onChange forza il ricaricamento della pagina con un get/post (**postback**)

Quindi:

- L'utente sceglie una provincia
- Viene invocata JSP con parametro provincia impostato al valore scelto dall'utente
- La pagina restituita contiene nella tendina dei comuni l'elenco di quelli che appartengono alla provincia scelta

Limiti del modello a ricaricamento di pagina

Quando lavoriamo con applicazioni desktop siamo abituati a un elevato livello di interattività:

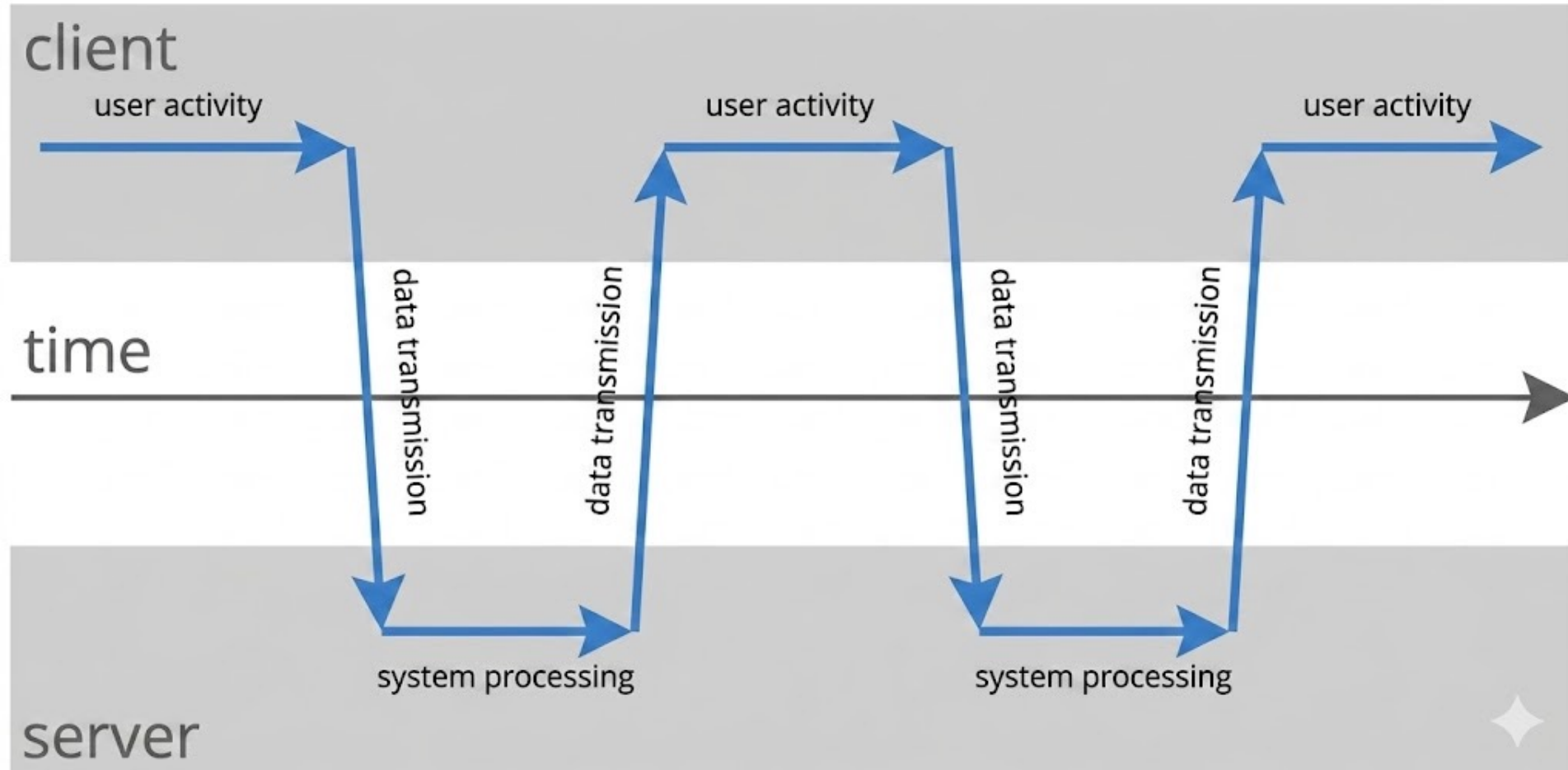
- applicazioni reagiscono in modo rapido e intuitivo ai comandi

Applicazioni Web tradizionali espongono invece un modello di interazione rigido

- Modello ***“Click, wait, and refresh”***
- È necessario refresh della pagina da parte del server per la gestione di qualunque evento (sottomissione di dati tramite form, visita di link per ottenere informazioni di interesse, ...)

È ancora un **modello sincrono**: l'utente effettua una richiesta e deve attendere la risposta da parte del server

Modello di interazione classico



Cos'è AJAX

Il modello di interazione **AJAX** è nato per superare queste limitazioni

AJAX sta per **Asynchronous JavaScript And XML** è uno stack di tecnologie, non una libreria

AJAX permette a una pagina Web di: leggere dati dal server DOPO essere stata caricata · aggiornare il proprio contenuto SENZA ricaricarsi · inviare dati al server IN BACKGROUND



READ

Recupera dati dal server dopo che la pagina è già stata caricata, senza interromperla



UPDATE

Aggiorna porzioni mirate del DOM senza un refresh completo della pagina



SEND

Invia dati al server in background, mentre l'utente continua a interagire

Lo "stack" tecnologico AJAX:

JavaScript & eventi

DOM API

HTML

CSS

XML o JSON

XMLHttpRequest

AJAX e asincronicità

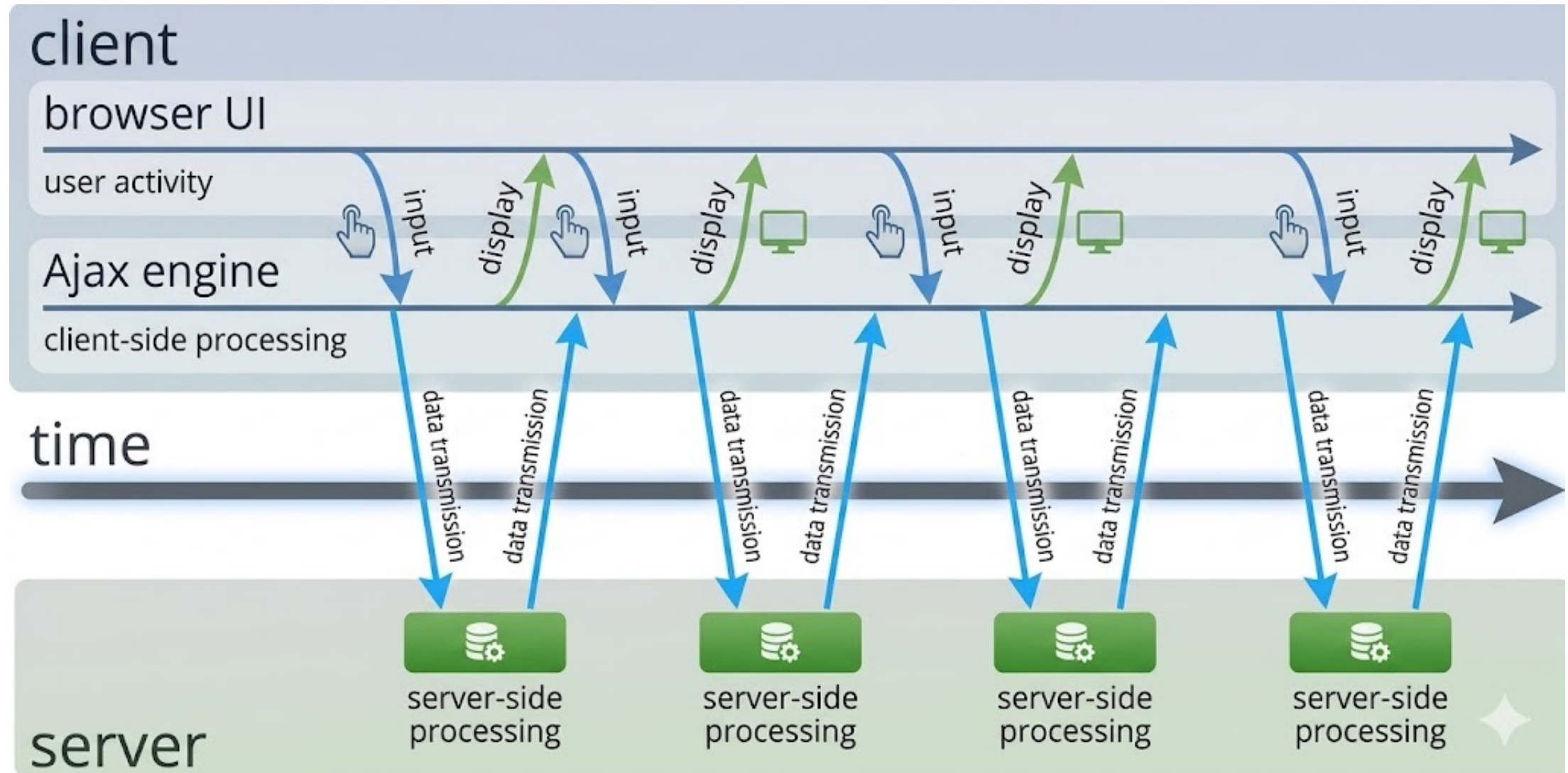
AJAX punta a supportare applicazioni user friendly con elevata interattività (si usa spesso il termine RIA - **Rich Interface Application**)

L'idea alla base di AJAX è quella di consentire agli script JavaScript di interagire direttamente con il server

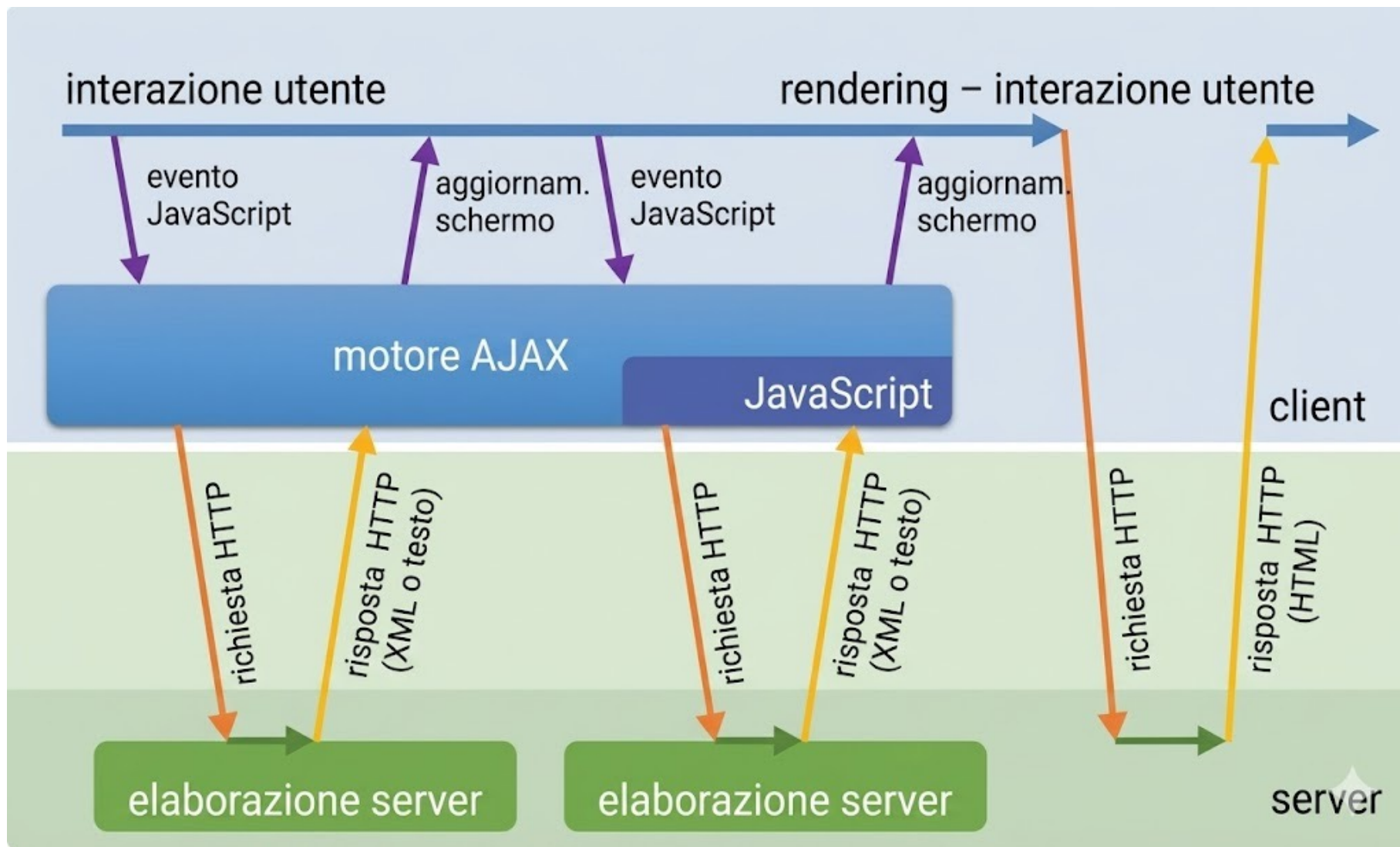
L'elemento centrale è l'utilizzo dell'oggetto JavaScript **XMLHttpRequest**

- Consente di ottenere dati dal server senza necessità di ricaricare l'intera pagina
- Realizza una comunicazione **asincrona** fra client e server: *il client non interrompe interazione con utente anche quando è in attesa di risposte dal server*

Modello di interazione con AJAX



Modello di interazione con AJAX



Tipica sequenza AJAX

1. Si verifica un evento determinato dall'interazione fra utente e pagina Web
2. L'evento comporta l'esecuzione di una funzione JavaScript in cui:
 - Si istanzia un oggetto di classe **XMLHttpRequest**
 - Si configura XMLHttpRequest: si associa una funzione di **callback**, si effettua una configurazione, ...
 - Si effettua chiamata asincrona al server
3. Il server elabora la richiesta e risponde al client
4. Il browser invoca la funzione di **callback** che:
 - elabora il risultato
 - aggiorna il DOM della pagina per mostrare i risultati dell'elaborazione

Un po' di storia

1998 Microsoft introduce XMLHTTP in Outlook Web Access (IE5)

2004 Gmail di Google sfrutta massicciamente XMLHttpRequest

2005 Jesse James Garrett conia il termine "AJAX"
Google Suggest diventa il caso emblematico

2006 Il W3C inizia la standardizzazione di XMLHttpRequest

2015 ES6 introduce le Promise; arriva la fetch() API moderna



XMLHttpRequest

È l'oggetto **XMLHttpRequest** che si occupa di effettuare la richiesta di una risorsa via HTTP a server Web. Può inviare eventuali informazioni sotto forma di parametri (come avviene con una form)

Può effettuare sia richieste GET che POST

Le richieste possono essere di tipo

✓ ASINCRONO

NON interrompe il flusso di esecuzione del codice JavaScript né le operazioni dell'utente sulla pagina. Il browser continua a rispondere all'utente. Una callback viene invocata al cambio di stato della richiesta. **È il modo CORRETTO di usare AJAX.**

✗ SINCRONO

Blocca completamente il thread principale del browser fino alla risposta. La pagina "si congela". **Vanifica il senso stesso di AJAX.**

Creazione di un'istanza: dettagli browser-specific

I browser recenti supportano **XMLHttpRequest** come oggetto nativo

Per creare un oggetto **XMLHttpRequest**, basta invocarne il costruttore:

```
var xhr = new XMLHttpRequest();
```

La gestione della compatibilità con browser "molto" vecchi complica un pò le cose, la esamineremo in seguito per non rendere difficile la comprensione del modello di interazione AJAX

Tipica sequenza di una chiamata AJAX

1

EVENTO

L'utente interagisce

Un click, un cambio di campo, un keyup...
Innesca una funzione JavaScript.



2

CHIAMATA

JS prepara e invia

Istanza XMLHttpRequest,
associa una callback,
chiama open() e send().



3

ELABORAZIONE

Il server lavora

Servlet/JSP elabora i parametri, accede al DB, prepara la risposta (XML/JSON/testo).



4

CALLBACK

Il browser reagisce

La risposta arriva: la callback elabora i dati e aggiorna il DOM. UI refreshata.

Tra fase 2 e fase 4 l'utente è LIBERO: può scrollare, digitare, fare altre azioni — il browser non lo blocca

Metodi di XMLHttpRequest

La lista dei metodi disponibili è diversa da browser a browser

In genere si usano solo quelli presenti in Safari (sottoinsieme più limitato, ma comune a tutti i browser che supportano AJAX):

open()

Inizializza la richiesta (metodo HTTP, URI, asincrono)

setRequestHeader()

Imposta header HTTP da inviare

send()

Invia effettivamente la richiesta al server

getResponseHeader()

Legge un singolo header HTTP dalla risposta del server

getAllResponseHeaders()

Legge TUTTI gli header HTTP della risposta

abort()

Interrompe immediatamente la richiesta in corso

Metodo open()

open() ha lo scopo di inizializzare la richiesta da formulare al server

Lo standard W3C prevede 5 parametri, di cui 3 opzionali:

open (method, uri [,async][,user][,password])

L'uso più comune per AJAX ne prevede 3, di cui uno comunemente fissato:

open (method, uri, true)

Dove:

- **method**: stringa e assume il valore "get" o "post"
- **uri**: stringa che identifica la risorsa da ottenere (URL assoluto o relativo)
- **async**: valore booleano che deve essere impostato come **true** per indicare al metodo che la richiesta da effettuare è di tipo asincrono

Metodi `setRequestHeader()` e `send()`

`setRequestHeader(nomeheader, valore)` consente di impostare gli header HTTP della richiesta da inviare

- Viene invocata più volte, una per ogni header da impostare
- Per una richiesta GET gli header sono opzionali
- Sono invece necessari per impostare la codifica utilizzata nelle richieste POST
- È comunque importante impostare header **connection** al valore **close** "opzione di chiusura" per il mittente di segnalare che la connessione verrà chiusa dopo il completamento della risposta)

`send(body)` consente di inviare la richiesta al server

- Non è bloccante se il parametro `async` di `open` è stato impostato a `true`.
- Prende come parametro una stringa che costituisce il body della richiesta HTTP

Esempi

GET

```
var xhr = new XMLHttpRequest();  
xhr.open("get", "pagina.html?p1=v1&p2=v2", true );  
xhr.setRequestHeader("connection", "close");  
xhr.send(null);
```

POST

```
var xhr = new XMLHttpRequest();  
xhr.open("post", "pagina.html", true );  
xhr.setRequestHeader("content-type",  
    "x-www-form-urlencoded");  
xhr.setRequestHeader("connection", "close");  
xhr.send("p1=v1&p2=v2");
```

Proprietà di XMLHttpRequest

Stato e risultati della richiesta vengono memorizzati dall'interprete JavaScript all'interno dell'oggetto durante la sua esecuzione

Le proprietà **XmlHttpRequest** comunemente supportate dai vari browser sono:

readyState

Intero (0-4): stato corrente della richiesta

STATO

onreadystatechange

Funzione di callback eseguita ad ogni cambio di readyState

EVENTO

status

Codice HTTP della risposta (200, 404, 500, ...)

ESITO

statusText

Descrizione testuale del codice HTTP

ESITO

responseText

Body della risposta come STRINGA

DATI

responseXML

Body della risposta parsato come documento XML

DATI

responseXML può essere null se il body non è XML ben formato. responseText è SEMPRE valorizzato.

Proprietà readyState

Proprietà in sola lettura di tipo intero che consente di leggere in ogni momento lo stato della richiesta

Ammette 5 valori:

- 0: **uninitialized** - l'oggetto esiste, ma non è stato ancora richiamato open()
- 1: **open** - è stato invocato il metodo open(), ma send() non ha ancora effettuato l'invio dati
- 2: **sent** - metodo send() è stato eseguito e ha effettuato la richiesta
- 3: **receiving** - la risposta ha cominciato ad arrivare
- 4: **loaded** - l'operazione è stata completata

Attenzione:

- *Questo ordine non è sempre identico e non è sfruttabile allo stesso modo su tutti i browser*
- **L'unico stato supportato da tutti i browser è il 4**

Proprietà onreadystatechange

Come si è detto l'esecuzione del codice non si blocca sulla ***send()*** in attesa dei risultati

Per gestire la risposta si deve quindi adottare un approccio a eventi

Occorre registrare una funzione di ***callback*** che viene richiamata in modo asincrono ad ogni cambio di stato della proprietà readyState

La sintassi è:

xhr.onreadystatechange = nomefunzione

xhr.onreadystatechange = function() {istruzioni}

*Attenzione: per evitare comportamenti imprevedibili l'assegnamento va fatto prima del ***send()****

Proprietà status e statusText

status contiene un valore intero corrispondente al codice HTTP dell'esito della richiesta:

200 OK (l'unico in base al quale i dati ricevuti in risposta possono essere ritenuti corretti e significativi)

403 Forbidden

404 Not Found

500 Internal Server Error

statusText contiene invece una descrizione testuale del codice HTTP restituito dal server...

Esempio:

```
if ( xhr.status != 200 ) alert( xhr.statusText );
```

Proprietà `responseText` e `responseXML`

Contengono i dati restituiti dal server

`responseText` stringa che contiene il body della risposta HTTP

- disponibile solo a interazione ultimata cioè: (**`readyState==4`**)

`responseXML` body della risposta convertito in documento XML (*se possibile*)

- anch'esso disponibile solo a interazione ultimata
- consente la navigazione del documento XML via JavaScript
- può essere **`null`** se i dati restituiti non sono un documento XML ben formato

Metodi `getResponseHeader()` e `getAllResponseHeaders()`

- Consentono di leggere gli header HTTP che descrivono la risposta del server
- Sono utilizzabili solo nella funzione di *callback*
- Possono essere invocati sicuramente in modo safe solo a richiesta conclusa (**`readyState==4`**)
- In alcuni browser possono essere invocati anche in fase di ricezione della risposta (**`readyState==3`**)

Sintassi:

- **`getAllResponseHeaders()`**
- **`getResponseHeader(header_name)`**

Ruolo della funzione di callback associata ad onreadystatechange

Viene invocata ad ogni variazione di **readyState**

Usa **readyState** per leggere lo stato di avanzamento della richiesta

Usa **status** per verificare l'esito della richiesta

Ha accesso agli header di risposta rilasciati dal server con

getAllResponseHeaders() e **getResponseHeader()**

Se **readyState==4** può leggere il contenuto della risposta con **responseText** e **responseXML**

Esempio

```
<!DOCTYPE html>
<html>
<body>

<div id="demo">
<h1>The XMLHttpRequest Object</h1>
<button type="button" onclick="loadDoc()">Change Content</button>
</div>

<script>
function loadDoc() {
  var xhttp = new XMLHttpRequest();
  xhttp.onreadystatechange = function() {
    if (this.readyState == 4 && this.status == 200) {
      document.getElementById("demo").innerHTML =
        this.responseText;
    }
  };
  xhttp.open("GET", "ajax_info.txt", true);
  xhttp.send();
}
</script>

</body>
</html>
```

The XMLHttpRequest Object

Change Content

AJAX

AJAX is not a programming language.

AJAX is a technique for accessing web servers from a web page.

AJAX stands for Asynchronous JavaScript And XML.

https://www.w3schools.com/xml/tryit.asp?filename=tryajax_first

Gestire la risposta XML

- AJAX è in grado di ricevere **documenti XML** accessibili tramite la proprietà **responseXML** dell'oggetto **XMLHttpRequest**
- É possibile elaborare i documenti XML ricevuti utilizzando l'API W3C DOM
- Il modo con cui operiamo su dati in formato XML è analogo a quello che abbiamo visto per ambienti Java
- Usiamo un parser e accediamo agli elementi del DOM XML di nostro interesse
 - L'oggetto **XMLHttpRequest** ha un in-built XML parser

```
var response = xhr.responseXML.documentElement;  
alert("Risposta: \n" + xhr.responseText);  
var result = response.getElementsByTagName("result")[0].firstChild.nodeValue;  
document.getElementById("datiCapoluogo").innerHTML = result;
```

Otengo il nodo testuale figlio
del primo nodo "result"

- Per visualizzare i contenuti ricevuti modifichiamo il DOM della pagina HTML

Esempio

Scegliamo un nome da una lista e mostriamo i suoi dati tramite Ajax

```
<html>
  <head>
    <script src="selectmanager_xml.js"></script>
  </head>
  <body>
    <form action=""> Scegli un contatto:
    <select name="manager"
      onchange="showManager(this.value)">
      <option value="Carlo11">Carlo Rossi</option>
      <option value="Anna23">Anna Bianchi</option>
      <option value="Giovanni75">Giovanni Verdi</option>
    </select></form>
    <b><span id="companyname"></span></b><br/>
      <span id="contactname"></span><br/>
      <span id="address"></span>
      <span id="city"></span><br/>
      <span id="country"></span>
    </body>
</html>
```

Lista di
selezione

Area in cui
mostrare i
risultati

Esempio

Ipotizziamo che i dati sui contatti siano contenuti in un database. Il server:

- riceve una request con l'identificativo della persona
- interroga il database
- restituisce un file XML con i dati richiesti

```
<?xml version='1.0' encoding='UTF-16' ?>  
<company>  
  <compname>Microsoft</compname>  
  <contname>Anna Bianchi</contname>  
  <address>Viale Risorgimento 2</address>  
  <city>Bologna</city>  
  <country>Italy</country>  
</company>
```

Esempio: selectmanager_xml.js

```
var xmlHttp;
function showManager(str)
{ xmlHttp=new XMLHttpRequest();
  var url="getmanager_xml.jsp?q="+str;
  xmlHttp.onreadystatechange=stateChanged;
  xmlHttp.open("GET",url,true);
  xmlHttp.send(null);
}
function stateChanged()
{ if (xmlHttp.readyState==4)
  {
    var xmlDoc=xmlHttp.responseXML.documentElement;
    var compEl=xmlDoc.getElementsByTagName("compname")[0];
    var comName = compEl.childNodes[0].nodeValue;
    document.getElementById("companymname").innerHTML=
      compName;
    ...
  }
}
```

Funzioni di Callback Multiple

- Il codice mostrato in precedenza non è in grado di svolgere più di un AJAX task (l'oggetto XMLHttpRequest è referenziato tramite una variabile globale)
- Se si devono eseguire più AJAX task in una web application, occorre creare una funzione per l'esecuzione dell'oggetto XMLHttpRequest e una funzione di callback per ciascun AJAX task
- La chiamata alla funzione dovrebbe contenere (almeno) l'URL e la funzione di callback da chiamare quando la risposta è pronta

```
// USO: due chiamate AJAX indipendenti
loadDoc("url-1", myFunction1);
loadDoc("url-2", myFunction2);

function loadDoc(url, cFunction) {
  const request = new XMLHttpRequest();
  request.onreadystatechange = function() { cFunction(this); }
  request.open("GET", url, true);
  request.send();
}

function myFunction1(request) { /* logica per url-1 */ }
function myFunction2(request) { /* logica per url-2 */ }
```

Vantaggi e svantaggi di AJAX

Più interattività, ma anche più complessità — un compromesso da gestire

+ Vantaggi

- Interattività comparabile al desktop
- Aggiornamenti mirati del DOM, senza refresh completo
- Riduzione del traffico di rete (si trasferiscono solo i dati)
- Tempo di attesa percepito molto minore
- L'utente continua a operare durante l'attesa
- Apre la strada alle SPA (Single Page Application)

− Svantaggi

- Si perde la linearità dell'interazione (più stati "impliciti")
- Possibile disorientamento dell'utente se manca feedback
- Le richieste possono durare troppo (timeout, errori di rete)
- Debug e test del codice JavaScript più complessi
- Manutenzione più difficile (logica frammentata in callback)
- Storico del browser e "deep linking" non funzionano in modo nativo
- SEO: i contenuti caricati dinamicamente potrebbero non essere indicizzati

Timeout e gestione dell'attesa

abort() consente l'interruzione delle operazioni di invio o ricezione

- non ha bisogno di parametri
- termina immediatamente la trasmissione dati

*Attenzione: non ha senso invocarlo dentro la funzione di **callback***

- Se readyState non cambia, il metodo non viene richiamato; readyState non cambia quando la risposta si fa attendere

Si crea un'altra funzione da far richiamare in modo asincrono al sistema mediante il metodo

setTimeout(funzioneAsincronaPerAbortire, timeOut)

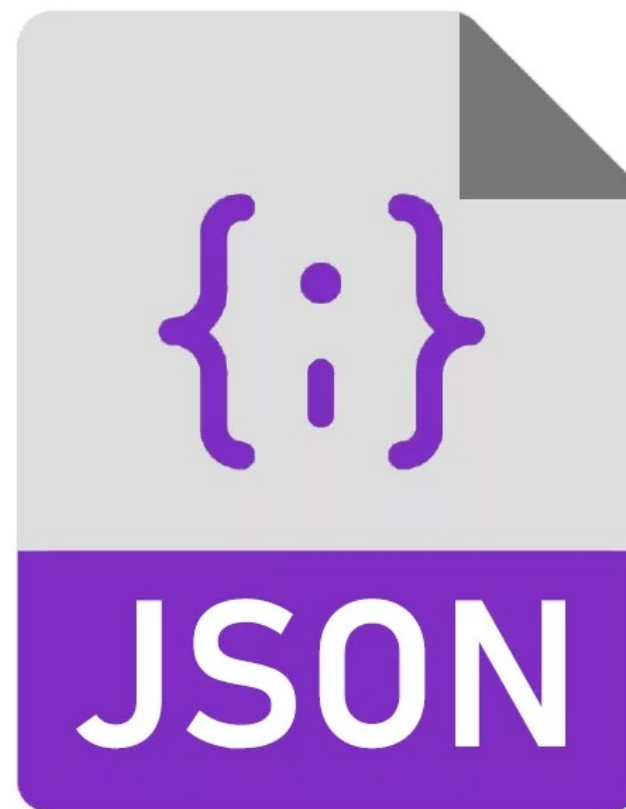
Al suo interno si valuta se continuare l'attesa o abortire l'operazione

```
// Schedula un controllo dopo 15 secondi
setTimeout(function() {
  if (request.readyState < 4) {      // non è ancora arrivata risposta
    request.abort();                // interrompi la richiesta
  }
}, 15000);                          // 15.000 ms = 15 secondi
```

XML vs. JSON



VS



XML è la scelta giusta?

Abbiamo però visto nell'esempio precedente che l'utilizzo di XML come formato di scambio fra client e server porta alla generazione e all'utilizzo di quantità di byte piuttosto elevate e non ottimizzate

- Difficile da leggere e da mantenere
- Oneroso in termini di risorse di elaborazione (*non dimentichiamo che JavaScript è interpretato*)

Esiste un formato più efficiente e semplice da manipolare per scambiare informazioni tramite AJAX?

- **La risposta è SI**; questo formato è nella pratica quello più utilizzato oggi

XML è ancora la scelta giusta?

XML

Verboso, tag di apertura/chiusura per ogni campo

```
<?xml version='1.0' ?>
<company>
  <compname>Microsoft</compname>
  <contname>Anna Bianchi</contname>
  <address>Viale Risorg. 2</address>
  <city>Bologna</city>
  <country>Italy</country>
</company>
```

JSON

Compatto, chiavi e valori, niente tag ridondanti

```
{
  "compname": "Microsoft",
  "contname": "Anna Bianchi",
  "address": "Viale Risorg. 2",
  "city": "Bologna",
  "country": "Italy"
}
```


JSON

JSON è l'acronimo di JavaScript Object Notation

- Formato per lo scambio di dati, considerato molto più comodo di XML
- Leggero in termini di quantità di dati scambiati
- Molto semplice ed efficiente da elaborare da parte del supporto runtime al linguaggio di programmazione (*in particolare per JavaScript*)
- Ragionevolmente semplice da leggere per un operatore umano
- È largamente supportato dai maggiori linguaggi di programmazione
- Si basa sulla notazione usata per gli object literal e gli array literal in JavaScript

Object Literal e Array Literal

In JavaScript è possibile creare un oggetto mediante un **object literal** creare un array utilizzando un **array literal** Possiamo anche avere **oggetti che contengono array** È infine possibile definire **array di oggetti**

1. Object literal

```
var Beatles = {  
  Paese: "Inghilterra",  
  AnnoFormazione: 1959,  
  TipoMusica: "Rock"  
};
```

2. Array literal

```
var Membri = [  
  "Paul", "John",  
  "George", "Ringo"  
];
```

Object Literal e Array Literal

In JavaScript è possibile creare un oggetto mediante un **object literal** creare un array utilizzando un **array literal** Possiamo anche avere **oggetti che contengono array** È infine possibile definire **array di oggetti**

3. Oggetti con array al loro interno

```
var Beatles = {  
  Paese: "Inghilterra",  
  AnnoFormazione: 1959,  
  TipoMusica: "Rock",  
  Membri: ["Paul", "John",  
           "George", "Ringo"]  
};
```

4. Array di oggetti (collezioni)

```
var Rockbands = [  
  {  
    "Nome": "Beatles",  
    "AnnoFormazione": 1959,  
    "Membri": ["Paul", "John", ...]  
  },  
  {  
    "Nome": "Rolling Stones",  
    "AnnoFormazione": 1962, ...  
  }  
];
```

Dalla sintassi JS alla stringa JSON

La sintassi JSON si basa su quella degli object e array literal di JavaScript. Una **stringa JSON** altro non è che la stringa equivalente ad un object o array literal JavaScript

```
{  
  "Paese":      "Inghilterra",  
  "AnnoFormazione": 1959,  
  "TipoMusica":  "Rock'n'Roll",  
  "Membri":      ["Paul", "John", "George", "Ringo"]  
}
```

**Object literal
JavaScript
(in memoria)**

↓ *serializzazione (JSON.stringify)*

```
'{"Paese":"Inghilterra","AnnoFormazione":1959,"TipoMusica":"Rock\'n\'Roll",  
  "Membri":["Paul","John","George","Ringo"]}'
```

```
// ↑ è una STRINGA: piccola, plain text, facile da spedire  
// e da rigenerare ovunque (qualsiasi linguaggio sa fare il parse)
```

**Stringa JSON
(trasmissibile
e via rete)**

Parser JSON

In passato, per ottenere un oggetto JavaScript a partire da una stringa JSON, si usava la funzione **eval()**. Uso di **eval()** presenta rischi di sicurezza: *stringa passata come parametro potrebbe contenere codice malevolo*

Oggi, si utilizzano parser appositi che traducono solo oggetti JSON e non espressioni JavaScript di qualunque tipo

JSON.parse(jsonString)

Stringa JSON → oggetto JavaScript

```
var json =  
'{"nome":"Anna","eta":23}';  
  
var obj = JSON.parse(json);  
  
console.log(obj.nome); // "Anna"  
console.log(obj.eta);  // 23  
  
// Lancia eccezione se la  
// stringa non è JSON valido
```

JSON.stringify(object)

Oggetto JavaScript → stringa JSON

```
var obj = {  
  nome: "Anna",  
  eta: 23,  
  hobby: ["musica", "sci"]  
};  
  
var json = JSON.stringify(obj);  
// '{"nome":"Anna","eta":23,...}'
```

Esempio

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>Create Object from JSON String</h2>
```

```
<p id="demo"></p>
```

```
<script>
```

```
let text = '{"employees":[' +  
'{"firstName":"John","lastName":"Doe" },' +  
'{"firstName":"Anna","lastName":"Smith" },' +  
'{"firstName":"Peter","lastName":"Jones" }]}';
```

```
const obj = JSON.parse(text);
```

```
document.getElementById("demo").innerHTML =  
obj.employees[1].firstName + " " + obj.employees[1].lastName;
```

```
</script>
```

```
</body>
```

```
</html>
```

Create Object from JSON String

Anna Smith

AJAX + JSON: lato server e lato client

Simile a quanto avviene con la ricezione di file XML, con due differenze, una lato server e una lato client

Lato Server:

- La servlet deve trasmettere la stringa JSON (non XML) nel body della risposta HTTP al client
- Occorre impostare il content type "application/json"
- Occorre un parser JSON (scarica da <https://github.com/stleary/JSON-java> la libreria **JSON-Java** e importala nel progetto Eclipse) per creare la stringa JSON

Lato client:

- Si converte la stringa JSON in un oggetto JavaScript usando **JSON.parse()**

* Un'altra libreria JSON molto usata in Java è GSON:

<https://mvnrepository.com/artifact/com.google.code.gson/gson>

AJAX + JSON: lato server e lato client

Lato SERVER (Java)

La servlet trasmette una stringa JSON nel body della risposta HTTP:

- Impostare il Content-Type: "application/json"
- Importare una libreria di serializzazione JSON
- Es. JSON-Java (stleary/JSON-java) — JSONObject, JSONArray
- Alternativa moderna: Google Gson o Jackson
- Costruire l'oggetto JSON e fare json.toString()

```
response.setContentType("application/json");
JSONObject json = new JSONObject();
json.put("result", risultato);
out.print(json.toString());
```

Lato CLIENT (JavaScript)

Nella callback della XHR, basta convertire la stringa ricevuta in un oggetto con JSON.parse():

- Si legge responseText (NON responseXML!)
- Si chiama JSON.parse() per ottenere un oggetto JS
- Si naviga l'oggetto con la dot notation (.prop)
- Si aggiorna il DOM normalmente

```
function handleCAP(request) {
    var response =
    JSON.parse(request.responseText);
    document.getElementById("datiCapoluogo")
        .innerHTML = response.result;
}
```


Riferimenti

W3Schools — AJAX Introduction [w3schools.com/xml/ajax_intro.asp](https://www.w3schools.com/xml/ajax_intro.asp)

W3Schools — JSON Tutorial [w3schools.com/js/js_json_intro.asp](https://www.w3schools.com/js/js_json_intro.asp)